Das können wir doch genauer beschreiben

Eine bessere Deutung verändert mehr als einen Tag im Text.

Nora stößt auf eine Briefstelle, die das bisherige Variantenmodell nur unzureichend erfasst. Sie möchte die Auszeichnung verändern. Nicht aus Freude am Umbau: Die neue Unterscheidung macht ihre wissenschaftliche Argumentation genauer.

Samir öffnet die Vorschau. Der neue Vorschlag betrifft auch Registerverweise, die Webdarstellung und den Satz. Frühere Bände wurden mit einer anderen Modellversion veröffentlicht. Die Frage ist nicht nur, ob Nora recht hat, sondern wie ihre Verbesserung in die Gesamtedition aufgenommen werden kann.

Die Kommission lässt einen begrenzten Versuch ausarbeiten. Das Team vergleicht Änderung nur für künftige Bände, rückwirkende Migration und parallele Modellversionen. Jede Variante bewahrt etwas und verursacht neue Arbeit.

Der beschlossene Modellwechsel wird in ODD, Beispielen und Handbuch mit Geltungsbereich dokumentiert. Alte Bandstände bleiben mit ihren damaligen Regeln nachvollziehbar.

> *„Das Schema darf nicht entscheiden, was ich wissenschaftlich sagen kann.“ — „Einverstanden. Aber wir müssen auch wissen, wie wir diese Aussage in zehn Jahren noch ausgeben.“*

Wie hilft Git?

Ein Versuchszweig hält die neue Modellierung getrennt erprobbar. Versionierte Daten, Schemata und Migrationsskripte machen Unterschiede nachvollziehbar. Ein Git-Tag benennt einen bestimmten Stand; ein Bandrelease benötigt zusätzlich die passenden Register- und Softwarestände.

Was gibt es zu bedenken?

Welche Altbestände sind betroffen? Wer prüft die semantische Richtigkeit einer Migration? Reicht ein neues Schema oder muss sich auch der Editionsbericht ändern?

Handlung

Eine repräsentative Testpassage wählen, Auswirkungen und Pflegekosten dokumentieren, fachlich und technisch prüfen und die Entscheidung mit ihrem Geltungsbereich festhalten.

Zuständig

Bandredaktion, wissenschaftlich-technische Redaktion und befugte Editionskommission

Institutionelle Frage: Wer finanziert die Verbesserung einschließlich der Arbeit an früheren Bänden?

Das Gutachten ist da. Und jetzt?

Eine fixierte Fassung ermöglicht ein Urteil. Sie beendet die Diskussion nicht.

Der Prüfstand ist verschickt, der Verlag hat ein Produktionsfenster vorgesehen. Dann trifft das externe Gutachten ein: Eine zentrale Textentscheidung sei noch nicht ausreichend begründet.

Nora übernimmt einen Hinweis, widerspricht einem zweiten und überarbeitet einen dritten Abschnitt. Eva organisiert, wer über die Antwort entscheidet und welche Veränderungen erneut begutachtet werden müssen. Die gutachtende Person soll weder das Git-System lernen noch die gesamte interne Arbeitsgeschichte sehen müssen.

Die Redaktion stellt einen lesbaren HTML- oder PDF-Abzug mit genauer Standkennung bereit. Rückmeldungen bleiben im vereinbarten vertraulichen Bereich. Das Ergebnis wird begründet dokumentiert; eine technische Prüfung allein entscheidet keine wissenschaftliche Streitfrage.

Das Handbuch legt fest, welcher Prüfstand und welche Regelversion zum Gutachten gehören. Die vertrauliche Rückmeldung liegt im gesonderten Reviewbereich; fachliche Antworten werden begründet dokumentiert.

> *„Wir können nicht jede Kritik einfach einbauen.“ — „Nein. Aber wir müssen zeigen können, wie wir sie geprüft haben.“*

Wie hilft Git?

Ein eindeutig markierter Prüfstand und der Vergleich zur Revision verbinden Rückmeldung und Bearbeitung. Geschützte Prüfpakete können aus versionierten Quellen erzeugt werden.

Was gibt es zu bedenken?

Wer darf das Gutachten lesen? Welche Anonymität ist vereinbart? Welche Änderung verlangt eine erneute Prüfung? Branches sind keine verlässliche Trennung von Leseberechtigungen.

Handlung

Prüfstand, Rückkanal, zuständige Instanz und Umgang mit Überarbeitungen vereinbaren; Terminfolgen mit dem Verlag abstimmen.

Zuständig

Gutachtende Person, Bandredaktion, Gesamtredaktion und Verlagslektorat

Institutionelle Frage: Wer trägt zusätzliche Reviewzeit und entscheidet bei bleibendem Dissens?

Die Daten wären für unsere Forschung ideal

Forschungsinteresse trifft auf eine bereits vereinbarte Publikationsstrategie.

Ein Forschungsteam möchte kommentierte Briefdaten für eine neue Netzwerkanalyse nutzen. Die Zusammenarbeit verspricht Erkenntnisse und macht die Edition früh sichtbar. Der betreffende Band ist aber noch nicht offen freigegeben.

Clara erinnert daran, dass der Verlag in Redaktion, Satz und Herstellung investiert. Die verzögerte digitale Bereitstellung war Teil der Vereinbarung. Eva will die Kooperation ermöglichen, ohne Zusagen zu übergehen.

Das Team trennt Metadaten, Text, Kommentar, Bilder und Verlags-PDF. Welche Nutzung ist bereits erlaubt, welche benötigt eine zusätzliche Vereinbarung? Eine Frist allein beantwortet diese Frage nicht. Der Fall setzt keine bestimmte Vertragslösung voraus.

Die Rechte-/Publikationsstelle nutzt das Freigabeverfahren aus dem Handbuch. Die Forschung an freigegebenen Briefdaten und ein späterer Zugang zur internen Entstehungsgeschichte sind unterschiedliche Anfragen.

> *„Die Daten sind doch etwas anderes als das Buch.“ — „Sind sie das? Wir müssen wissen, was wir tatsächlich miteinander vereinbart haben.“*

Wie hilft Git?

Nach geklärter Freigabe kann ein kuratiertes Datenpaket exakt bezeichnet und im eigenen öffentlichen Repo veröffentlicht werden. Private Geschichte und vertrauliche Rückmeldungen werden nicht mitexportiert.

Was gibt es zu bedenken?

Welche Gegenstände umfasst der Vertrag? Welche Drittmaterialien haben eigene Bedingungen? Was ist nur zugänglich, was auch nachnutzbar? Wer darf verbindlich entscheiden?

Handlung

Konkrete Vereinbarungen prüfen, Datenarten trennen, gegebenenfalls zusätzlichen Zugang aushandeln und jede Freigabe nachvollziehbar dokumentieren.

Zuständig

Projektleitung, zuständige Rechteberatung, berechtigte Stellen und Verlag

Institutionelle Frage: Welche Unterstützung ermöglicht Forschung und eine tragfähige Publikation zugleich?

Band sieben, bitte noch einmal

2055 ist der alte Host verschwunden. Der wissenschaftliche Zusammenhang soll bleiben.

Eine neue Arbeitsgruppe möchte einen älteren Band erneut ausgeben. Die damalige Datenbank ist nicht mehr im Betrieb, der ursprüngliche Git-Host existiert nicht mehr. Ein Buchband und ein Datenpaket sind erhalten.

Im Paket liegt ein Manifest: Es nennt Textstand, Schema, Register-Snapshot und Ausgabeprogramm. Die Gruppe prüft, ob diese Bestandteile tatsächlich vorhanden sind. Eine Liste allein ersetzt die Dateien nicht.

Wo sich die frühere Ausgabe rekonstruieren lässt, dokumentiert das Team den Weg. Wo etwas fehlt, kennzeichnet es die Lücke. Eine neue Ausgabe erhält einen eigenen verantworteten Stand. Die frühere Publikation wird nicht nachträglich als unverändert reproduziert ausgegeben.

Im Archivpaket findet die Gruppe auch das damalige Handbuch und die Profilbeschreibung. Erst damit wird verständlich, welche Prüfungen und Übergaben zu diesem Bandstand gehörten.

> *„Wir haben die Texte.“ — „Gut. Aber haben wir auch das Register, auf das diese Texte verweisen?“*

Wie hilft Git?

Eine übertragbare Versionsgeschichte bewahrt Dateistände und ihre Beziehungen. Sie kann den Neustart erleichtern, ersetzt aber weder DB-Snapshots noch Kontext, Rechteentscheidungen und verfügbare Transformationen.

Was gibt es zu bedenken?

Was ist tatsächlich erhalten? Lassen sich Ausgabe und Verweise testen? Welche Plattformdaten wurden separat übernommen? Wer darf die neue Ausgabe freigeben?

Handlung

Archivpakete auf Vollständigkeit prüfen, Wiederherstellung regelmäßig erproben, fehlende Bestandteile dokumentieren und Übergaben institutionell absichern.

Zuständig

Infrastrukturträger, Archivdienst und neue wissenschaftliche Redaktion

Institutionelle Frage: Welche wiederkehrenden Tests hätten die Lücken früher sichtbar gemacht?

ZWISCHEN SITUATION 04 UND 05

Die Edition lernt mit jedem Band dazu

Über Jahrzehnte bleiben die Arbeitsabläufe einer Edition nicht stehen. Wiederkehrende Schritte lassen sich mit Erfahrung besser organisieren; zugleich machen wachsende Register und immer mehr Querverweise neue Lösungen nötig. Die Redaktion erprobt Routinen, übernimmt Bewährtes und bessert nach. Jede eingeführte Änderung erhält eine passende Handbuchfassung; Nacharbeit an älteren Beständen wird als eigener Auftrag dokumentiert.

Auch technische Fortschritte fließen ein: OCR kann Vorlagen vorerschließen, LLM können etwa Vorschläge für Annotationen liefern. Solche Ergebnisse werden fachlich geprüft. Neue Regeln sollen außerdem auf ältere Daten anwendbar werden, ohne bereits geleistete Arbeit unbemerkt zu überschreiben.

Git hilft, Daten, Regeln und Verarbeitungsskripte in passenden Ständen zusammenzuhalten. Die Redaktion kann eine Änderung zunächst getrennt erproben, Ergebnisse vergleichen und nachvollziehen, welche Altdaten schon nachgezogen wurden. Ob die Verbesserung fachlich stimmt und die Übergaben funktionieren, muss sie weiterhin prüfen. Genau darum geht es in der nächsten Situation.

Die Satzprobe war doch schon geprüft

Eine beliebte neue Oberfläche erleichtert die Kommentierung. Samir entdeckt, wie ihr Rückweg bereits geprüfte Arbeit wieder verändert.

„Das haben wir doch längst korrigiert!“

Mal stimmt ein Abstand nicht, mal ein Strich, mal ein Registerverweis. Zunächst wirkt jeder Befund für sich klein. Zusammen bringen sie die Buchproduktion ins Stocken. Der Verlag hat eine Werbekampagne bezahlt; ausgerechnet im Albrecht-Jubiläumsjahr droht wieder ein Band auszufallen. Clara macht dem Satzdienstleister Vorwürfe.

Der widerspricht: Die angelieferten Daten sähen schlampig aus. Unzählige Kleinigkeiten würden im Satz sogar stillschweigend verbessert. „Kann den Editoren niemand erklären, was ein geschütztes Leerzeichen ist?“

Die Redaktion ist empört. Sie hat Abstände und Stricharten sorgfältig markiert und die betreffenden Stellen unmittelbar danach geprüft. Nora erinnert sich genau an eine Korrektur, die in der Fahne wieder falsch ist. Aus Sicht der Redaktion muss der Fehler beim Satz hineingekommen sein.

Samir hört sich den Streit an. Er verspricht, sich die Vorgeschichte der betroffenen Stellen anzusehen: Vielleicht lässt sich herausfinden, wann und auf welchem Weg die Fehler zurückgekommen sind.

Samirs erste Spur: die späten Bearbeitungen

Samir erklärt, wie er begonnen hat: „Ich habe alle erst spät korrigierten Stellen zusammengestellt. Dann habe ich nachgesehen: Was waren die letzten Änderungen vor der Korrektur? Und welche Bearbeitungen liegen zwischen dem ersten Auftreten des Fehlers und seiner Behebung? Was haben diese Wege gemeinsam?“

In seiner Auswahl liegen die zugehörigen Zeitstempel mehrheitlich spät in der Nacht. Er zeigt einige Einträge. „Und überhaupt: Je näher die Deadline rückt, desto weiter reichen die Einträge in die Nacht. Zuerst dachte ich, wir hätten ein Konzentrationsproblem und sollten vielleicht etwas an unseren Biorhythmen ändern.“ Einige lachen.

Ein Ausschnitt aus Samirs Ansicht

04.06.	**00:27**	Kommentar mit Register abgeglichen
06.06.	**00:37**	Überarbeitete Kommentare zurückgespielt
08.06.	**02:30**	Registerabgleich abgeschlossen
09.06.	**03:35**	Korrekturen für die nächste Fahne übernommen

Beispielhafte Commit-Zeitstempel aus der Fallgeschichte.

„Aber das war es nicht“, fährt er fort. „Die späten Zeitstempel haben die Ursache nicht erklärt. Ich musste mir die Stellen selbst und ihre Vorgeschichte genauer ansehen.“ Mit git blame findet er einen Einstieg in die zugeordnete Änderung; mit git show und dem Vergleich zweier Fassungen liest er weiter zurück. Zeichen, Annotation und Prüfabzug betrachtet er zusammen.

Die Redaktion erinnert sich richtig: Die besonderen Abstände und der richtige Strich waren schon korrekt gespeichert. Samir wechselt zwischen dieser Nahlektüre und dem Vergleich vieler Fälle. Im Überblick findet er einen gemeinsamen Bearbeitungsweg. In den einzelnen Fassungen sieht er, was auf diesem Weg verloren geht.

git blame: Welche Änderung steckt hinter dieser Zeile?

git blame ordnet jeder Zeile einer ausgewählten Dateifassung die Änderung zu, die sie zuletzt verändert hat. Samir bekommt so eine Commit-Kennung und kann die zugehörige Änderung öffnen.

Das ist ein Einstieg in die Vorgeschichte: Eine spätere Neuformatierung kann ebenfalls als letzte Änderung erscheinen. Der Befehl erklärt weder die fachliche Ursache noch eine persönliche Verantwortung. Deshalb liest Samir mit git show und Fassungsvergleichen weiter.

```
git blame -L 42,48 -- kommentare.xml
```

Zeigt die Änderungszuordnung für die Zeilen 42 bis 48 der Datei kommentare.xml.

Git-Dokumentation: blame ↗

git show: Was wurde in diesem Schritt verändert?

git show öffnet einen ausgewählten Commit: seine Beschreibung und bei einem gewöhnlichen Commit die Änderungen gegenüber dem Vorgänger. Samir sieht, welche Dateien und Textstellen gemeinsam verändert wurden.

Mit der Commit-Kennung aus git blame verfolgt er so den Rückimport. Für den direkten Vergleich zweier ausgewählter Fassungen nutzt er zusätzlich git diff. Das fachliche Lesen der XML-Stellen bleibt Teil der Untersuchung.

```
git show a1b2c3d -- kommentare.xml
```

a1b2c3d steht hier beispielhaft für eine Commit-Kennung. Angezeigt wird die Änderung an kommentare.xml.

Git-Dokumentation: show ↗

Git: blame ↗ · Git: show ↗ · Git: Unterschiede vergleichen ↗

Die neue Kommentaroberfläche: eine große Erleichterung

Seit einiger Zeit gibt es eine neue Oberfläche für Kommentare, die mit dem Register abgeglichen werden müssen. Links steht der lesbare Kommentar, rechts erscheinen passende Personen-, Orts- und Werkdatensätze. Die Bearbeitenden können Treffer vergleichen und die richtige Verknüpfung auswählen, ohne ständig zwischen XML-Dateien und Registerlisten zu wechseln.

Alle bestätigen den Gewinn. Die Arbeit geht leichter von der Hand, Verwechslungen lassen sich unmittelbar klären. „Diese Oberfläche möchten wir auf jeden Fall behalten“, sagt Nora. Niemand möchte zur mühsamen früheren Arbeitsweise zurück.

Genau in diesem erfolgreichen neuen Weg findet Samir aber die Ursache. Bei seiner Einbindung ins Projekt werden Kommentare in die Arbeitsansicht übertragen und anschließend zurückgespielt. Der Rückimport vereinfacht bestimmte besondere Abstände und Striche. Außerdem führt die Anbindung bestehende Datumsauszeichnungen nicht verlustfrei mit: Sie rekonstruiert sie anschließend mit der Zahlenerkennung aus dem zurückgegebenen Text.

Damit hängen zwei Dinge zusammen, die in der bequemen Oberfläche unsichtbar bleiben: Ein sorgfältig gesetzter Bereichsstrich wird verändert – und die daraus erneut berechnete Auszeichnung sagt nun etwas anderes aus. Die Oberfläche ist eine echte Hilfe. Ihr Rückweg braucht eine Reparatur.

DIE SCHLÜSSELSTELLE IM WORKFLOW

Bequemer kommentieren. Beim Zurückspielen geht Information verloren.

Der Fortschritt ist unstrittig: Kommentar und passende Registerdaten stehen nebeneinander. Der Fehler steckt in der Einbindung dieses neuen Arbeitswegs.

KOMMENTAR · LESBARE ARBEITSANSICHT

Albrechts Briefwechsel entwickelte sich 1848–1849 in enger Verbindung mit seinem Bruder.

Empfänger und Registerbezüge direkt am Text abgleichen.

PASSENDE REGISTERDATEN

- Person: Albrechts Bruder
- Zeitraum: 1848–1849
- Abgleich direkt am Kommentar

> *„Diese Oberfläche möchten wir auf jeden Fall behalten.“*

1. Geprüfte TEI-Daten

Marker und Zeitraum sind bereits korrekt gespeichert.

2. Lesbare Kommentaransicht

Nur für Kommentare mit Registerabgleich. Die fachliche Bearbeitung wird leichter.

3. Fehlerhafter Rückweg

Zeichen werden vereinfacht; Datumsauszeichnungen aus dem Text neu berechnet.

Vor dem Export richtig – nach dem Rückimport verändert. Eine frühere Prüfung kann diesen späteren Verlust nicht verhindern.

Schematische Ansicht des im Fall verwendeten Arbeitswegs.

Warum es nur manche Kommentare trifft

Die übrigen Kommentare werden weiterhin im bisherigen XML-Werkzeug bearbeitet. Nur die Kommentare mit Registerabgleich nehmen den Weg durch die neue Oberfläche. Deshalb tauchen die Fehler nicht überall auf.

Auch unter diesen Kommentaren bleiben viele unauffällig: Wer keinen der betroffenen Abstände oder Striche enthält, kann den problematischen Rückweg ohne sichtbaren Unfall passieren. Erst das Zusammentreffen von Bearbeitungsweg und relevanten Zeichen löst die beschriebene Kette aus.

Samir kann nun die vielen Einzelbefunde erklären. „Achtet genauer auf eure Zeichen“ hatte die Redaktion längst getan. Die Marker waren geprüft worden – vor dem späteren Schritt, der sie wieder vereinfachte.

Aus einem Strich wird eine Registerkaskade

An einem kleinen Beispiel wird die Folge sichtbar. Im Kommentar steht „1848–1849“, ausgezeichnet als zusammenhängender Zeitraum. Ein automatisierter Schritt erkennt solche Zeitangaben und fügt eine Annotationsschicht für das Register ein. Dabei nutzt die projektinterne Zahlenerkennung auch den Bereichsstrich. Nach dem Rückweg steht dort „1848-1849“: Beim erneuten Aufbau der Schicht erzeugt die Regel nun zwei einzelne Jahresnennungen. Die vorherige Zeitraum-Auszeichnung wird nicht erhalten.

Die Regel erzeugt formal erlaubte, aber sachlich anders zugeordnete Daten. Registereinträge und das Web-Findemittel werden daraus abgeleitet. Zugleich können verlorene geschützte Abstände den Umbruch ändern. In der Fahne findet jemand den falschen Strich und repariert ihn. Ein zuvor schon für den Satz freigegebenes Register passt nun nicht mehr zum korrigierten Text- und Ausgabestand.

Nicht nur die Fahne ist betroffen. Auch das separate Web-Findemittel, Registerauszüge und bereits vorbereitete Registerbände müssen auf die betroffenen Änderungen geprüft und gegebenenfalls neu erzeugt werden. Stabile IDs verhindern manche Verweisverschiebung, aber keine falsche zeitliche Annotation oder einen veralteten Seitenverweis.

Die Abhängigkeit liegt hier in den vereinbarten Projektregeln und Ausgabewegen. Weder ein gewöhnlicher Strich noch TEI führen von sich aus zu dieser Interpretation. Genau diese Regeln muss Samir mitprüfen.

So wird aus einem Zeitraum etwas anderes

Vor dem Bearbeitungsweg: ein Zeitraum

```
<date from="1848" to="1849">1848–1849</date>
```

from und to beschreiben Anfang und Ende des zusammengehörenden Zeitraums.

Nach dem fehlerhaften Rückweg und der Neuauszeichnung

```
<date when="1848">1848</date>-<date when="1849">1849</date>
```

when bezeichnet hier jeweils ein einzelnes Jahr. Die Verbindung als Zeitraum fehlt. Beide Fragmente können als Struktur erlaubt sein; die Aussage ist nicht dieselbe.

Eine bloße Ersetzung des sichtbaren Strichs innerhalb der ursprünglichen date-Auszeichnung würde from und to nicht löschen. Im Fall führt die fehlerhafte Anbindung die ursprüngliche Annotation nicht verlustfrei mit und baut sie aus dem vereinfachten Text neu auf. Deshalb muss Samir auch Export, Import und Zahlenerkennung zusammen prüfen.

XML-Ausschnitte im TEI-Kontext. TEI erklärt date und Datumsattribute ↗

Ein selektiver Umweg, mehrere abhängige Ausgaben

Die neue Oberfläche erleichtert den Registerabgleich. Nur ein Teil der Kommentare durchläuft sie. Der Rückweg verändert bestimmte Marker und rekonstruiert Datumsannotation aus dem Text; genau diese Kombination löst den Fehler aus.

> **Marker und Zeitraum stimmen**
>
> „1848–1849“ ist als Zeitraum erkannt. Typografische Marker sind geprüft; das Register passt zu diesem Datenstand.

> **Der Rückweg verändert die Zeichen**
>
> Nur registerbezogene Kommentare nehmen den Weg durch die neue Oberfläche. Beim Rückimport wird „1848–1849“ zu „1848-1849“. Die frühere Datumsauszeichnung wird nicht erhalten; die anschließende Projektregel erzeugt zwei einzelne Jahresnennungen statt des Zeitraums.

> **Richtig korrigiert – später wieder falsch**
>
> Eine Satzkorrektur stellt den Strich wieder her; Register, Web-Findemittel und Registerbände müssen erneut abgestimmt werden. Bei einer späteren berechtigten Kommentarkorrektur führt derselbe Registerweg den Zeichenverlust erneut ein.

Gezielte Kontrolle: Rückweg reparieren; nach dem Abgleich die betroffenen Marker, Annotationen und Ausgaben prüfen. Korrekturen im maßgeblichen Bestand halten und abhängige Stände gemeinsam neu erzeugen.

Lukas korrigiert einen Sachfehler – und löst den Rückfall aus

Besonders deutlich wird es an einem Kommentar von Lukas. Der falsche Strich war bereits entdeckt und repariert worden. Die Korrektur war in den maßgeblichen Datenbestand zurückgeführt; das Register war erneut geprüft und für den Satz freigegeben.

Dann liest Lukas die nächste Fahne und entdeckt einen wirklichen eigenen Fehler: Er hat den Empfänger eines Briefs verwechselt. Er öffnet den Kommentar in der vertrauten neuen Oberfläche, berichtigt den Empfänger und wählt den passenden Personeneintrag aus dem Register. Eine richtige und notwendige Korrektur.

Nach dem Zurückspielen ist der schon einmal reparierte Bereichsstrich wieder zum einfachen „-“ geworden. Lukas hat die betreffende Zeitangabe gar nicht angefasst. Trotzdem berechnet die Anbindung ihre Annotation neu. Zeitraum und Registerzuordnung stimmen erneut nicht; die Kaskade erreicht wieder Satz, Web-Findemittel und Registerausgaben.

„Aber ich habe doch nur den Empfänger korrigiert!“, sagt Lukas. Genau das macht den Fall für Samir so aufschlussreich. Nicht die neue fachliche Korrektur war falsch. Sie hat denselben fehlerhaften Rückweg noch einmal ausgelöst. Der scheinbar wieder unachtsame Verursacher hat den Mechanismus unabsichtlich reproduziert.

Die richtige Korrektur nimmt erneut den falschen Weg

1. **Bereits repariert**

   Strich und Zeitraum stimmen wieder; Register und Satzstand sind geprüft.

2. **Lukas entdeckt einen Sachfehler**

   Er korrigiert den Briefempfänger und gleicht den Kommentar erneut mit dem Register ab.

3. **Unberührte Zeitangabe verändert**

   Der Rückimport vereinfacht den Strich und erzeugt die Datumsauszeichnung erneut.

4. **Die Kaskade ist zurück**

   Register, Satz, Web-Findemittel und Registerbände müssen wieder zusammen geprüft werden.

Der aufschlussreiche Unterschied: Lukas korrigiert den Inhalt richtig. Seine Korrektur löst den fehlerhaften Übertragungsweg erneut aus.

Was für ein Workflow-Problem ist das?

Samir unterscheidet den verlustbehafteten Übertragungsweg, die ungünstige Abfolge von Prüfung und späterer Veränderung, die Weitergabe in abhängige Ausgaben und die wiederholte Reparatur nur sichtbarer Symptome. Solche Schwächen können zusammenwirken; nicht jeder Fehler hat dieselbe Ursache.

Die sofortige Gegenmaßnahme liegt nach dem Rückimport: Die Redaktion prüft dort gezielt Zeichen, Annotation und Ausgabe. Diese Zusatzkontrolle wird sofort im Handbuch festgehalten. Parallel wird die fehlerhafte Anbindung des Kommentartools überarbeitet – damit die geprüften Inhalte künftig den ganzen Weg unverändert überstehen.

Fachliche Korrekturen gehen in den maßgeblichen Datenbestand zurück. Daraus werden betroffene Annotationen und Ausgaben kontrolliert neu abgeleitet. Ein Freigabestand dokumentiert gemeinsam Text, Register, Regeln und Ausgabeweg. Stille Satzreparaturen ohne Rückführung dürfen nicht zum dauerhaften zweiten Datenbestand werden.

PRAXIS VERBESSERN UND FESTHALTEN

Aus BUG-58 wird Handbuch 1.1

Bisher
Fassung 1.0: Zeichen und Zeitraum vor der Übergabe prüfen.

Neu freigegeben
Fassung 1.1 · H06: Nach dem Rückimport Zeichen UND Zeitraumauszeichnung kontrollieren; Registerannotation und abhängige Ausgaben aus demselben Stand prüfen.

Die Buchproduktion braucht jetzt eine verlässliche Übergabe. Samir schlägt deshalb eine sofort umsetzbare Zusatzkontrolle vor: Nach dem Rückimport werden die betroffenen Zeichen und Zeitraumauszeichnungen geprüft. Nora legt die fachlichen Kriterien fest. Eva gibt die Übergangsregel frei und erklärt sie in der Redaktionsrunde; im Handbuch steht sie ab sofort als H06.

Gleichzeitig beauftragt Eva die Überarbeitung des Kommentartools. Samir und die Entwicklung untersuchen Export und Rückimport, erhalten die ursprüngliche Annotation und ergänzen einen Regressionstest. Das braucht mehr Zeit als die zusätzliche Prüfung. Bis der neue Weg abgenommen ist, arbeitet die Redaktion mit der Übergangsregel weiter.

Später spielt Lukas seine Empfängerkorrektur im überarbeiteten Tool durch: Die unangetastete Zeitangabe bleibt erhalten. Nora und Samir nehmen den verbesserten Hin-und-zurück-Weg fachlich und technisch ab. Die Zusatzkontrolle hat die Zwischenzeit abgesichert; die Tooländerung beseitigt die Ursache. Beides wurde veranlasst und umgesetzt.

Damit wird auch die nächste Handbuchänderung fällig. Das Team prüft, welche zusätzliche Kontrolle nach der bewährten Tooländerung noch nötig ist. Eine Übergangsregel bleibt nicht allein deshalb für immer bestehen, weil sie einmal sinnvoll war.

1	2	3	4	5
Befund und Änderungsvorschlag	Fachliches und technisches Review	Freigabe und neuer Test	Bekanntgabe und Anwendung	Nächste gemeinsame Prüfung

Das gemeinsame Handbuch ansehen ↑

Welche Messgrößen wären für die Edition sinnvoll?

DevOps-Messgrößen liefern Anregungen, keine fertigen Leistungsmaßstäbe für Editionsarbeit. Diese Vorschläge benötigen zusätzliche fachliche Klassifikation und Prüfdaten.

Späte Fehler nach einem Arbeitsschritt

Bestätigte Fehler, die erst nach einer Übergabe oder Satzprobe auffallen, im Verhältnis zum vergleichbar geprüften Umfang. Gleichbleibende Fehlerklassen und Prüfdichte sind wichtig.

Zeit von Meldung bis geprüfter Behebung

Beginn: dokumentierte Fehlermeldung. Ende: fachlich geprüfte Korrektur. Wartezeiten, Schwierigkeit und Zuständigkeit mitführen; weder individuelle Arbeitszeit noch automatisch eine Fristverletzung.

Erneut geöffnete Korrekturen

Welche zunächst erledigten Meldungen müssen wieder geöffnet werden? Zusammengehörige Folgefehler gruppieren, damit dieselbe Kette nicht als viele unabhängige Probleme erscheint.

Änderung ohne erneute Bezugsprüfung

Nach einer relevanten Text-/Kommentaränderung: Gibt es einen Nachweis, dass Ziel und aktuelle Ausgabe zusammen geprüft wurden? Prüfnachweise sind zusätzliche Daten, keine aus Git ablesbare Wahrheit.

Gemeinsam geänderte Bereiche

Welche Dateien oder Bereiche werden häufig zusammen geändert? Das kann eine Abhängigkeit oder bloß eine gemeinsame Formatierung anzeigen. Eine auffällige Kopplung ist ein Anlass zum Nachsehen, kein Fehlerbeweis.

Änderungen an TEI-Strukturen

Welche Elemente, Attribute und Beziehungen werden tatsächlich geändert? Mechanische Formatierung und Massenänderungen getrennt betrachten. Ein strukturbezogener Vergleich erkennt noch nicht automatisch einen Bedeutungswandel.

Persistenz formaler Probleme

Wie lange bleibt ein erkanntes Schema- oder Referenzproblem bestehen? Prüfzeitpunkte und die jeweils gültigen Regelversionen dokumentieren; eine später verschärfte Regel nicht unbemerkt auf frühere Arbeit als damaligen Fehler zurückprojizieren.

Kodierung vergleichbarer Fälle

Wo weichen vergleichbare Stellen von den dokumentierten Konventionen ab? Materialtyp und Projektphase berücksichtigen. Fachlich begründete Ausnahmen von unbeabsichtigten Inkonsistenzen unterscheiden.

DORA erklärt Messgrößen der Softwareauslieferung ↗. Eine Korrekturzeit im Editionsprozess ist nicht automatisch eine DORA-Messgröße.

NEUER SCHWERPUNKT · DATENSCHUTZ

Wenn die Analyse den Blick auf Menschen lenkt

Bisher ging es um Fehleranalyse, Monitoring und die Verbesserung der Abläufe. Jetzt stellt sich eine andere Frage: Welche personenbezogenen Einsichten entstehen dabei – und wer darf sie wofür verwenden?

„Trotzdem sollten wir über unsere Arbeitszeiten sprechen“

Die technische Ursache, die schnelle Absicherung und die Überarbeitung des Tools sind besprochen. Samir kommt auf seinen Einstieg zurück: „Trotzdem: Sollten wir vielleicht einmal über unsere Arbeitszeiten sprechen? Vor der Deadline wurde es schon ziemlich spät. Und wir könnten regelmäßig verfolgen, wie lange Korrekturen liegen bleiben.“

„Über die Deadline gern“, sagt eine Kollegin. „Aber ich teile mir meine Zeit selbst ein. Manchmal arbeite ich lieber nachts und mache dafür am Nachmittag etwas anderes. Ich möchte jetzt nicht an Sollzeiten für Kommentare gemessen werden. Wir sind schließlich nicht in der Softwareindustrie.“

Eva nickt: „Wann du am besten arbeitest, ist deine Sache. Wenn wir vor jeder Abgabe solche Nächte brauchen, sollten wir aber über Termine und Unterstützung sprechen. Das gehört zu meiner Verantwortung.“

„Einverstanden. Unsere Vereinbarung beschreibt die Fehlersuche. Gehört so eine regelmäßige Auswertung auch dazu?“ Samir schüttelt den Kopf. „Nein. Und aus den nächtlichen Einträgen hätte ich keine Konzentrationsgeschichte machen sollen. Bevor wir daraus eine Routine machen, müssen wir Frage und Grenzen gemeinsam klären.“ Seine Zeitstempel sind kein Arbeitszeitnachweis.

Gemeinsame Verantwortung braucht geklärte Maßstäbe

Die Mitarbeitervereinbarung war beim Einstieg erläutert worden. Sie regelt Beiträge, Zugänge und den Schutz der Arbeitsgeschichte. Samirs neue Auswertung ist dadurch nicht pauschal erlaubt. Das Team grenzt den technischen Auftrag ab: Welche Übergabe verändert die Daten? Eine regelmäßige Beobachtung von Tageszeiten gehört nicht dazu.

Es empfiehlt sich, auch die Maßstäbe für Fortschritt und Qualität gemeinsam zu entwickeln. In der Softwareentwicklung ist eine solche gemeinsame Verständigung etwa in Scrum üblich: Das Team prüft seinen Arbeitsablauf und vereinbart Verbesserungen. Für die Edition könnten verlässliche Übergaben, fachlich geprüfte Bearbeitungsstände und weniger wiederholte Nachkorrekturen zählen. Welche Messwerte dazu passen, lässt sich aus den Aufgaben der Edition ableiten; individuelle Sollzeiten für wissenschaftliche Kommentare folgen daraus nicht.

Ein langlebiges Projekt wird seine Abläufe weiter verändern müssen. Gemeinsam geklärte Zwecke, Zugriffe und Grenzen erleichtern diese Anpassungen. So können Samirs präzise Einblicke sowohl technische Verbesserungen als auch ein sachliches Gespräch über Planung und Entlastung ermöglichen. Rechtliche und ethische Grenzen bleiben dabei verbindlich.

Scrum Guide: gemeinsame Verbesserung in der Retrospektive ↗

Zuständig: Redaktion, Mitarbeitende und gegebenenfalls ihre Vertretungen, Koordination, technische und rechtlich zuständige Stellen

Institutionelle Frage: Welche Regeln, Ressourcen und gemeinsamen Maßstäbe braucht das Projekt, damit Prozesssteuerung und das Gespräch über Belastung fair stattfinden können?